

KidLisp: A Minimal DSL for Human–Machine Co-Creation of Generative Art

Paper type: Short Paper

Anonymous Authors

Abstract

We describe KIDLISP, a minimal Lisp dialect embedded in a browser-based creative computing platform, designed so that humans and large language models (LLMs) produce programs that each can read, modify, and remix. The language exposes 118 domain-specific primitives across 12 categories, has no file I/O, networking, or general string manipulation, and omits user-defined procedures. Every program receives a short alphanumeric code, is stored content-addressed, is executable at a URL, and can be embedded inside other programs as a first-class composition primitive. We report on a deployed corpus of over 16,000 programs by 59 authors created over twenty-two months. We argue that the constraints that make KIDLISP tractable for novices also make it unusually legible to LLMs, and that the combination of a constrained vocabulary with social-by-default distribution produces co-creative workflows qualitatively different from those supported by general-purpose creative coding environments.

Introduction

Computational creativity research increasingly treats human–machine *co-creation* as a central setting (Kantosalo and Toivonen 2016; Davis et al. 2015): a creative system’s value depends not only on what it can autonomously produce but on how well it scaffolds a human collaborator’s work. Generative language models have made this question newly practical. Anyone with a chat interface can ask an LLM for a Processing sketch or a p5.js program. But two problems recur. First, general-purpose code produced by an LLM is frequently longer than the human collaborator can read, let alone modify. Second, the outputs live in isolated files on isolated machines; the collaboration does not accumulate.

We describe KIDLISP, a minimal domain-specific Lisp embedded in AESTHETIC COMPUTER, a browser-based creative computing platform. KIDLISP starts from a different question than prior creative coding languages: *what is the smallest language that can produce interesting generative art, and does that same minimality make it legible to an LLM?* A complete animated program can be a single line:

```
(ink rainbow) (repeat 100 i
  (circle (wiggle width) (wiggle height)
    10))
```

The language ships with 118 primitives, no user-defined procedures, no file I/O, no networking, no general string manipulation, and deterministic rendering seeded from each program’s short code. Every program is stored content-addressed in a MongoDB collection, given a short code (e.g. `cow`), executable at `/ $cow`, and embeddable in another program via the `$code` syntax. The platform has accumulated over 16,000 programs by 59 authors.

We claim that KIDLISP’s constraints—chosen originally to make a language small enough for children to learn through social participation (Resnick et al. 2009)—turn out to also make it a good substrate for human–LLM co-authorship: both parties produce programs the other can read.

Language Design

KIDLISP is an S-expression language. Each program is a sequence of expressions evaluated top-to-bottom, producing side effects on a Canvas 2D context, a WebGL context, a Web Audio graph, or some combination thereof.

Design principles

1. **Every expression draws.** Valid KIDLISP produces visible output. There is no `main`, no imports, no boilerplate.
2. **Flat over deep.** Programs compose by *embedding* other programs, not by defining abstractions. This keeps code readable at a glance.
3. **Safe by construction.** No file I/O, no networking, no mutation of external state. Programs are sandboxed by the language’s lack of dangerous primitives, so arbitrary user- or LLM-submitted code can execute without review.
4. **Social by default.** Every program gets a short code and a URL. Sharing is the distribution mechanism, not an afterthought.

Function categories The 118 built-ins span 12 categories (Table 1). Each primitive has a mnemonic name and a small, positional arity.

Timing syntax Temporal operators schedule evaluation without explicit loops or callbacks: `2.5s` (after 2.5s), `1s...` (every second), `3s!` (once at 3s), `30f` (after 30

Category	n	Examples
Graphics	9	wipe, ink, line, box, circle
Transforms	11	scroll, zoom, spin, blur, suck
Math	14	+, sin, cos, random, wiggle
Colors	19	CSS names + rainbow, zebra
System	9	width, height, frame, fps
3D	8	cube, form, trans, move
Audio	6	mic, melody, overtone
Control	7	def, if, repeat, once, later
Text	4	write, type, paste
Input	3	pen, touch
Composition	4	embed, layer, fade
Data	4	let, list, get, set
Total	118	

Table 1: KIDLISP built-in functions by category.

frames). These compose with any expression and form the language’s entire animation model.

Chaos mode Invalid or random text input triggers artistic visual output rather than error messages. A confidence-scored detector examines word-recognition rate, special-character ratio, and parenthesis balance to classify input as code or chaos; chaotic input produces deterministic visuals seeded from the input. There are, effectively, no error states visible to the user—a design choice that matters for first-time users and is indirectly relevant to LLM output, which may arrive partially malformed.

What it omits Recursion, user-defined functions beyond `def` for named values, string manipulation, file I/O, networking, mutable data structures, exception handling. These omissions are the design. Removing abstraction mechanisms keeps programs flat: a reader understands any program top-to-bottom without tracing a call stack.

Storage, Composition, Distribution

Each program is stored as a document keyed by a content-addressed SHA-256 hash of its trimmed source and a short, human-meaningful code. The short code is derived from program content—extracting dominant function names, color keywords, or structural patterns—falling back to random generation when all inferred codes are taken. This produces codes like `cow` for a program with cow-like shapes rather than arbitrary alphanumeric strings.

A single REST endpoint serves the corpus: POST stores a program (deduplicating by hash); GET with a code returns a single program; GET with a comma-separated list returns up to 50 programs for embedded-layer resolution during evaluation; GET with a `recent` flag paginates a feed; GET with a `stats=functions` flag returns weighted function-usage counts across the corpus. Authentication is optional with a three-second timeout so anonymous creation is always fast.

Embedding The `$code` syntax inside a program fetches that program from storage (or cache), evaluates it into an offscreen buffer, and composites the result onto the parent canvas. This creates a directed acyclic composition graph. Users build complex visuals by layering simple programs, not by writing complex single programs.

```
; A 3D cube grid with two embedded programs
(wipe black)
(def n 6)
(repeat (* n n) i
  (ink (hop 0.1 rainbow blue white))
  (form (trans (cube i)
    (move (* (% i n) 3) 0 -20)
    (scale 1.5) (spin 0 1 0))))
($27z)
($cow)
```

In our corpus the maximum observed embedding depth is 7 layers.

Co-Creation with LLMs

Three properties of the language make it unusually tractable as an LLM output target.

Vocabulary is closed and small. Any generated program draws from a fixed set of 118 primitives. An LLM-generated program with an unknown symbol is immediately flagged; there is no equivalent of silently importing a typo’d library. The constrained surface area also means the full language reference fits comfortably into an LLM’s context window with room for examples.

Programs are short. Because abstraction is unavailable, a working program is necessarily under a few dozen lines. A human collaborator can read the entire program end-to-end, change one number, and observe the result. This makes the conversation with an LLM tight: the model proposes, the human edits, the model can re-read the edited program in its next turn without losing context.

Output is safe to execute. Lack of dangerous primitives means code produced by an LLM can be evaluated immediately in the shared browser runtime without trust decisions. The feedback loop from “LLM produces code” to “human sees rendered result” is one click; no container, no sandbox configuration, no API key.

Together, these turn KIDLISP into a shared notation for human–LLM co-creation rather than a language that an LLM writes *for* a user. Users report a characteristic workflow: ask the model for a sketch; trim it; recombine it with an embedded program already in the corpus via `$code`; save the result, which itself becomes available as a remixable building block. The corpus then grows as a library of co-authored fragments, indexed by short code.

This workflow aligns with Kantosalo and Toivonen’s (2016) characterization of co-creative systems as those that treat the human and machine as peers in the production of an artifact, and with Davis et al. (2015)’s argument that the *interaction design* of a co-creative system is as consequential as its generative capacity.

Evaluation

We report deployment data collected from the live platform.

Metric	Value
Total programs created	16,174
Unique authors	59
Programs per author (median)	12
Programs per author (top quartile)	180+
Max observed embedding depth	7 layers
Platform registered users	2,798
Evaluator source size (lines)	15,161

Table 2: Adoption metrics, March 2026.

Function usage analytics The `?stats=functions` endpoint scans up to 5,000 programs sorted by hits, parses each source to extract function calls, and returns counts weighted by program popularity. The most-used functions are `wipe`, `ink`, `repeat`, and `circle`—confirming that drawing and iteration, rather than the more elaborate transforms and composition primitives, anchor most practice. Boden’s (1992) distinction between *exploratory* and *transformational* creativity is visible in the corpus: most programs sit in the exploratory region (recombinations of common primitives), with rarer transformational work appearing at the embedding frontier, where users compose previously unrelated programs via `$code`.

Observations **Composition over abstraction.** Users embed rather than abstract. The most-referenced programs are simple visual primitives (gradients, shapes, patterns) that serve as building blocks. **Exploration over engineering.** Programs are typically under ten lines. Users iterate by saving new programs rather than editing existing ones, creating divergent exploration paths visible in the storage timestamps. **Social learning.** The most prolific authors began by viewing and remixing existing programs; the short-code system makes discovery accidental.

Following Ritchie’s (2007) empirical criteria, the system exhibits high *quantity* and measurable *novelty*—each short-code-keyed program is, by content-addressed construction, distinct from every other—while *quality* is harder to assess automatically and is left to the social signal of the `hits` counter.

Related Work

Creative coding. Processing (Reas and Fry 2007) and p5.js (McCarthy and Processing Foundation 2015) are the dominant tools; Sonic Pi (Aaron 2016) targets live musical coding. All require managing a project structure and a general-purpose language.

Live coding and the browser. Hydra provides browser-based live visual coding via JavaScript method chaining but without persistent corpus or social infrastructure. Strudel (Roos and McLean 2023) brings TidalCycles-style pattern coding to the browser; like KIDLISP, it is installation-free, but it targets rhythmic pattern composition rather than generative visual art.

Educational languages. Scratch (Resnick et al. 2009) demonstrated that social sharing drives engagement in educational programming. KIDLISP inherits this insight but uses text-based Lisp syntax and deliberately positions LLMs as a first-class peer in authoring.

Minimal Lisp implementations. Van Engelen (van Engelen 2022) showed a usable Lisp in 99 lines of C. KIDLISP’s evaluator is considerably larger (15,161 lines) because the bulk of the code implements drawing, audio, and composition primitives rather than Lisp semantics.

Co-creative systems. Davis et al. (2015) and Kantosalo and Toivonen (2016) frame creativity support as a design problem in interaction between human and machine agents; our contribution reads as an argument for minimal DSLs as a particularly tractable substrate for that interaction under modern LLM capabilities.

Conclusion

A minimal, browser-native, content-addressed DSL supports a productive community of generative-art practitioners and, as a side effect of the same constraints, an unusually tractable human–LLM co-creation loop. The language’s 118-primitive vocabulary fits in a model context window; its syntactic flatness keeps programs human-readable end-to-end; its lack of dangerous primitives makes generated code safe to execute without sandboxing decisions. The resulting corpus—over 16,000 timestamped, attributed programs with an explicit composition graph—is, in addition to an artwork platform, a dataset for studying how non-expert users and LLMs jointly converge on creative output.

Author Contributions

[Anonymised for review.]

References

- Aaron, S. 2016. *Sonic Pi: The Live Coding Music Synth for Everyone*. <https://sonic-pi.net>.
- Boden, M. 1992. *The Creative Mind*. London: Abacus.
- Davis, N.; Hsiao, C.-P.; Singh, K. Y.; Li, L.; Moningi, S.; and Magerko, B. 2015. Drawing apprentice: An enactive co-creative agent for artistic collaboration. In *Proceedings of the 2015 ACM SIGCHI Conference on Creativity and Cognition*, 185–186. ACM.
- Kantosalo, A., and Toivonen, H. 2016. Modes for creative human-computer collaboration: Alternating and task-divided co-creativity. In *Proceedings of the Seventh International Conference on Computational Creativity (ICCC)*, 77–84.
- McCarthy, L., and Processing Foundation. 2015. p5.js: A JavaScript library for creative coding. <https://p5js.org>.
- Reas, C., and Fry, B. 2007. *Processing: A Programming Handbook for Visual Designers and Artists*. MIT Press.
- Resnick, M.; Maloney, J.; Monroy-Hernández, A.; Rusk, N.; Eastmond, E.; Brennan, K.; Millner, A.; Rosenbaum, E.; Silver, J.; Silverman, B.; and Kafai, Y. 2009. Scratch:

Programming for all. In *Communications of the ACM*, volume 52, 60–67.

Ritchie, G. 2007. Some empirical criteria for attributing creativity to a computer program. *Minds and Machines* 17:76–99.

Roos, F., and McLean, A. 2023. Strudel: Live coding patterns on the web. In *Proceedings of the International Conference on Live Coding (ICLC)*.

van Engelen, R. 2022. tinylisp: A lisp interpreter in 99 lines of C. <https://github.com/Robert-van-Engelen/tinylisp>.